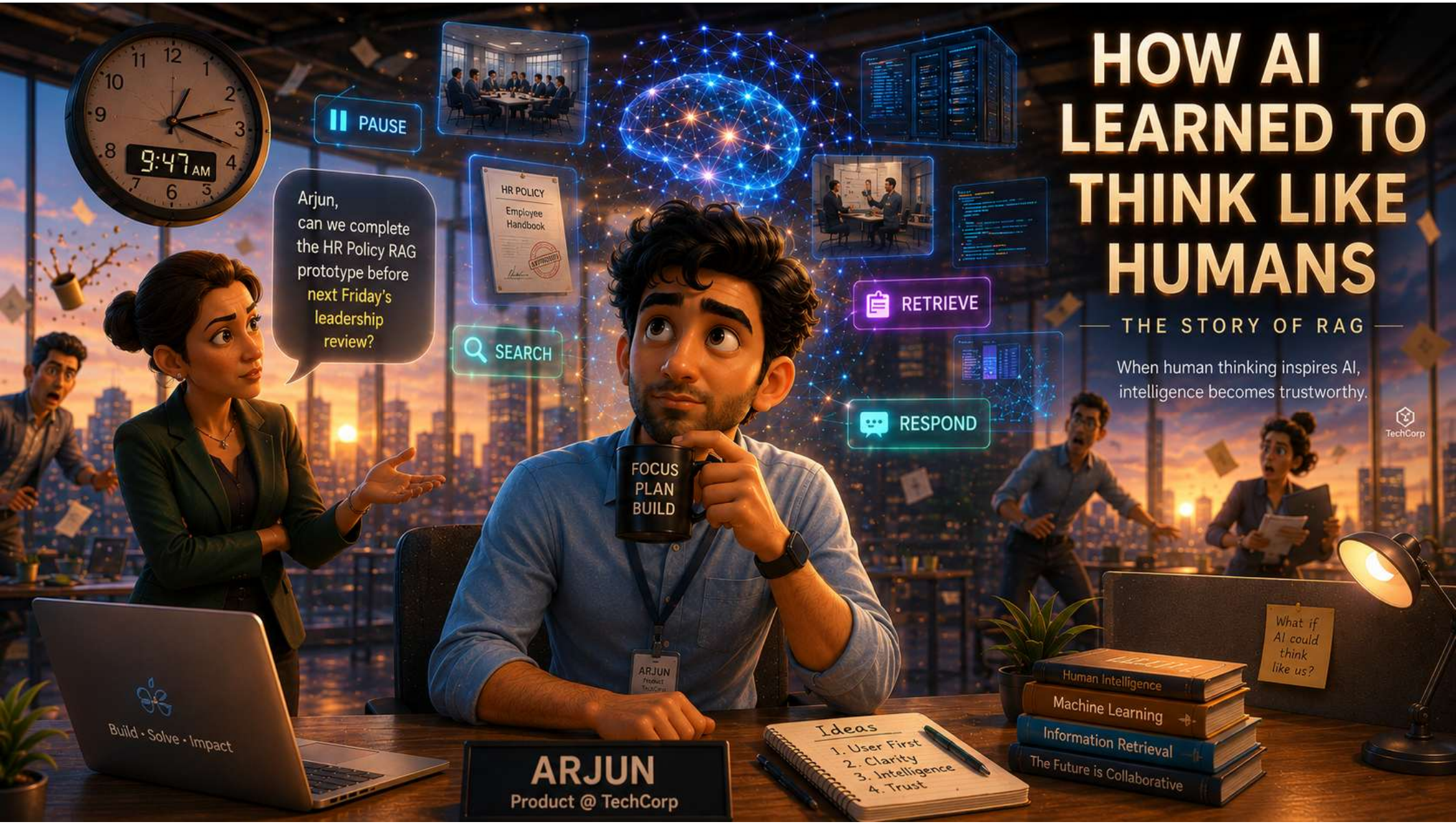

RAG
END TO END
Concepts Made Simple
Hands-on Practice
Deploy Real Applications
UNDERSTANDING • THEORY • PRACTICAL • ACCURACY CHECK • DEPLOYMENT
Build Real-World RAG Applications with Confidence
1 UNDERSTANDING
What is RAG?
Why RAG?
Components
Use Cases
Architecture Deep Dive
2 THEORY
Embeddings
Vector Databases
Similarity Search
Context Retrieval
Prompt Engineering
3 PRACTICAL
Build RAG Pipeline
Ingest Documents
Chunking & Embeddings
Retrieve Context
Generate Answers
4 ACCURACY CHECK
Relevance Evaluation
Faithfulness Check
Hallucination Detection
Answer Grading
Metrics & Techniques
5 DEPLOYMENT
API Development
Web App (UI)
Performance Optimization
Monitoring & Logging
Production Deployment
DATA SOURCES
CHUNKING
EMBEDDINGS
VECTOR DATABASE
RETRIEVAL
LLM GENERATION
ANSWER
END TO END PROJECT
Real-world RAG Application
CLEAN CODE
Well Structured & Modular
DEPLOY ANYWHERE
Cloud / On-Prem / Hybrid


HOW AI LEARNED TO THINK LIKE HUMANS
THE STORY OF RAG
When human thinking inspires AI, intelligence becomes trustworthy.
TechCorp
PAUSE
9:47 AM
Arjun, can we complete the HR Policy RAG prototype before next Friday's leadership review?
HR POLICY
Employee Handbook
SEARCH
RETRIEVE
RESPOND
FOCUS
PLAN
BUILD
ARJUN
Product @ TechCorp
Build · Solve · Impact
Ideas
1. User First
2. Clarity
3. Intelligence
4. Trust
What if AI could think like us?
Human Intelligence
Machine Learning
Information Retrieval
The Future is Collaborative

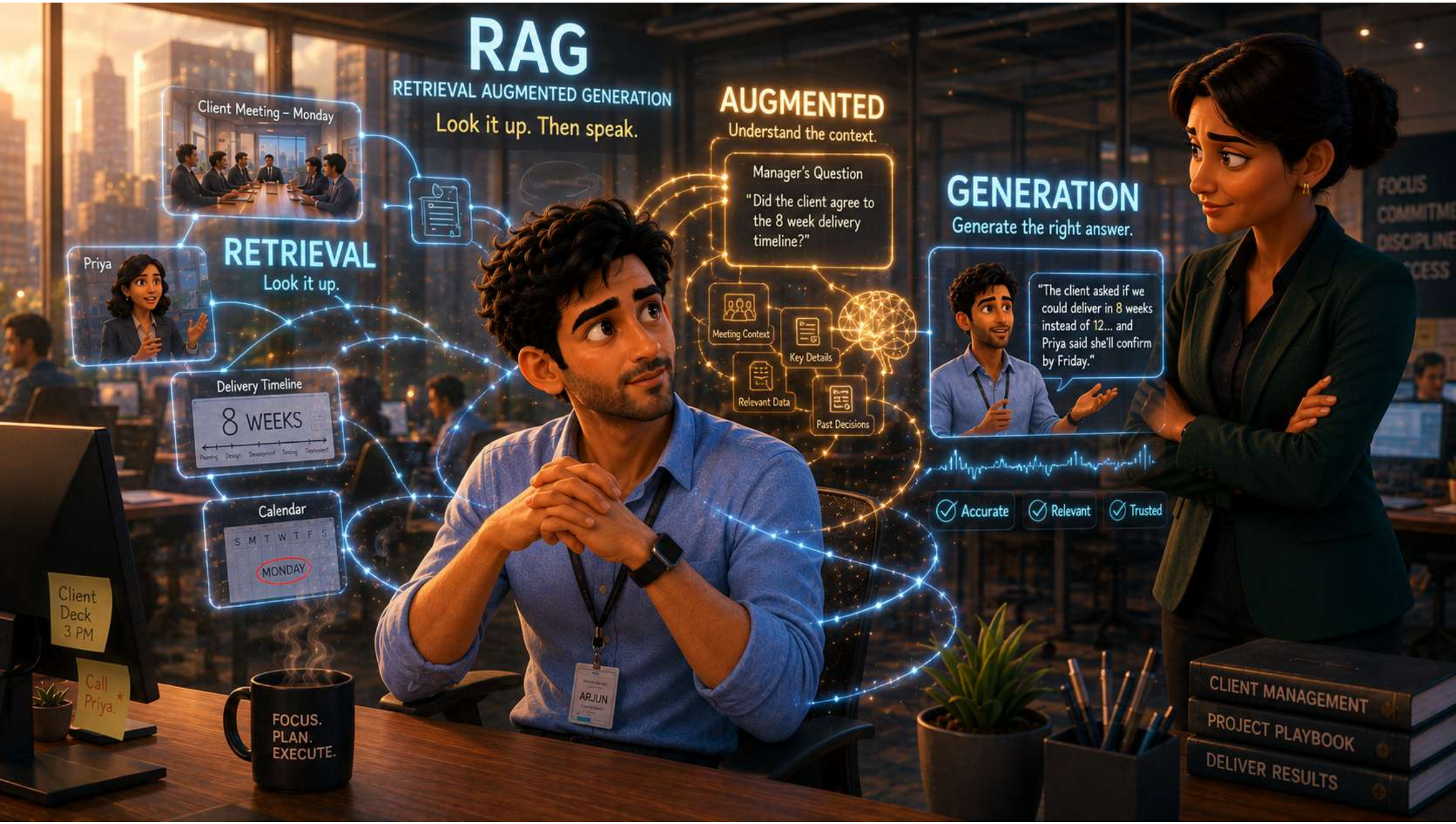

Hey Arjun, what delivery timeline did the client agree to in Monday's meeting?
FOCUS. PLAN. EXECUTE.
ARJUN
FOCUS
COMMITMENT
DISCIPLINE
SUCCESS
Focus Creates Clarity
Retrieve Reality
THOUGHTFUL TODAY. IMPACT TOMORROW.
DATA
KNOWLEDGE
WISDOM
IMPACT


Timeline?
8 weeks instead of 12
FOCUS.
PLAN.
EXECUTE.
Client Deck 3 PM
Call Priya.
CLIENT MANAGEMENT
PROJECT PLAYBOOK


Client Deck 3 PM
Call Priya.
FOCUS. PLAN. EXECUTE.
ARJUN
FOCUS
COMMITMENT
CLIENT MANAGEMENT
PROJECT PLAYBOOK
DELIVER RESULTS


Monday Client Meeting
timeline
Timeline Discussion
Client Presentation
Calendar
MONDAY
delivery
Delivery Timeline
8 WEEKS
3rd Floor - Meeting Room
MEETING ROOM 3A
8 weeks
Project Plan
deadline
Client Deck 3 PM
Call Priya.
FOCUS. PLAN. EXECUTE.
ARJUN
FOCUS
COMMITMENT
DISCIPLINE
CLIENT MANAGEMENT
PROJECT PLAYBOOK

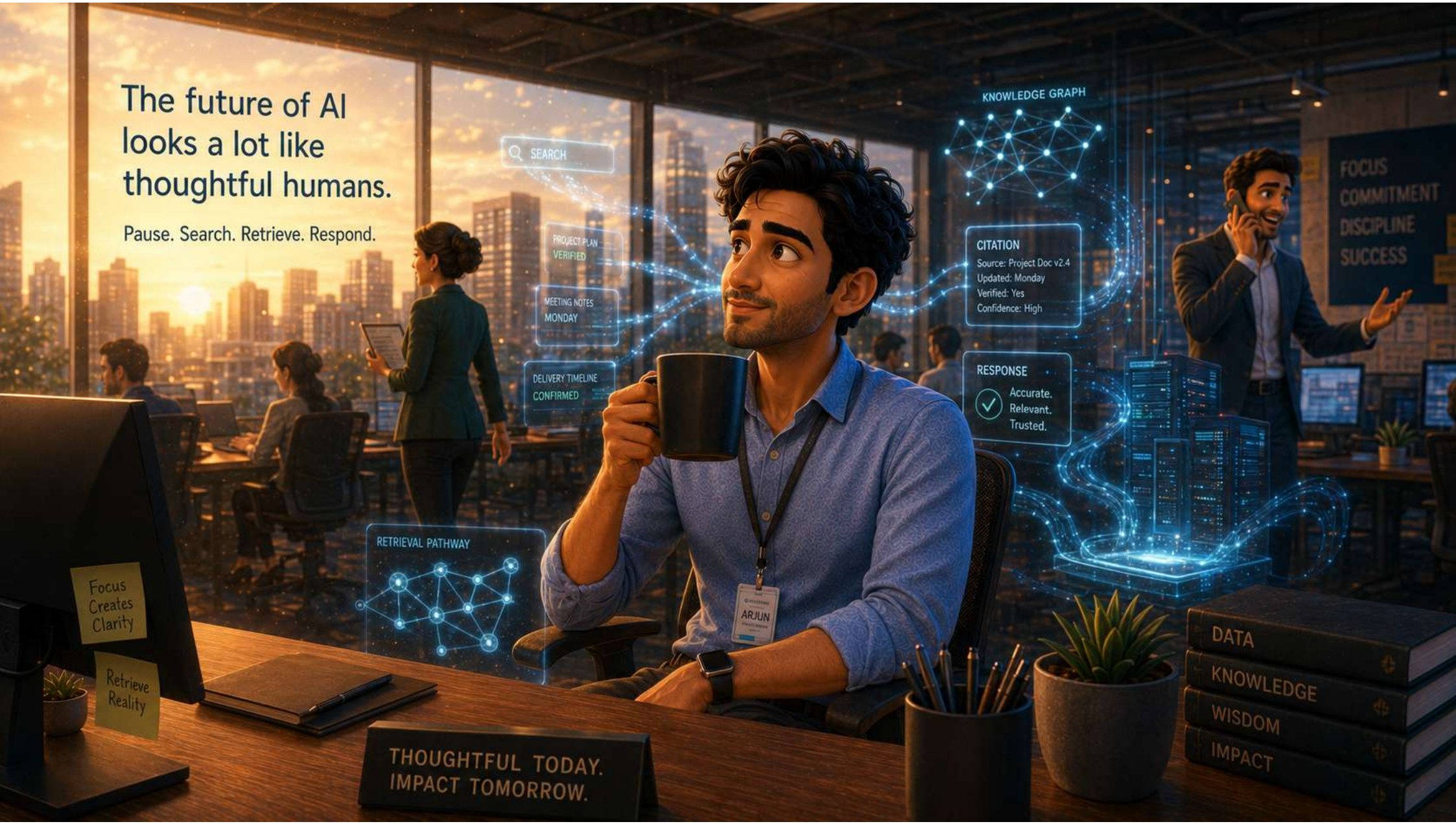

The client asked if we could deliver in 8 weeks instead of 12... and Priya said she'll confirm by Friday.
Delivery Timeline
8 WEEKS
Planning
Design
Development
Testing
Deployment
Priya
Calendar
S M T W T F S
MONDAY
Timeline Notes
Delivery – 8 Weeks
Key Modules
Client Approval
Testing Cycle
Client Concern
Chai Break Discussion
Client Deck 3 PM
Call Priya.
FOCUS. PLAN. EXECUTE.
ARJUN
FOCUS
COMMITMENT
DISCIPLINE
CLIENT MANAGEMENT
PROJECT PLAYBOOK
DELIVER RESULTS


RAG
RETRIEVAL AUGMENTED GENERATION
Look it up. Then speak.
Client Meeting – Monday
Priya
RETRIEVAL
Look it up.
Delivery Timeline
8 WEEKS
Calendar
S M T W T F S
MONDAY
AUGMENTED
Understand the context.
Manager's Question
"Did the client agree to the 8 week delivery timeline?"
Meeting Context
Key Details
Relevant Data
Past Decisions
GENERATION
Generate the right answer.
"The client asked if we could deliver in 8 weeks instead of 12... and Priya said she'll confirm by Friday."
Accurate
Relevant
Trusted
Client Deck 3 PM
Call Priya
FOCUS. PLAN. EXECUTE.
ARJUN
CLIENT MANAGEMENT
PROJECT PLAYBOOK
DELIVER RESULTS


What did we order at that restaurant last month?
What was the bug we fixed last sprint?
Did the electrician come on Tuesday?
Meeting Notes
- 8 Weeks
- Timeline
- Client Ask
Priya
Delivery Timeline
8 WEEKS
Monday
MONDAY
FOCUS
COMMITMENT
DISCIPLINE
SUCCESS
Client Deck 3 PM
Call Priya.
FOCUS. PLAN. EXECUTE.
ARJUN
We've been doing RAG our whole lives.
CLIENT MANAGEMENT
PROJECT PLAYBOOK
DELIVER RESULTS


RESULTS
TEAMWORK
INTEGRITY
15%
ARJUN
Focus
> Speed
I DON'T
GUESS.
I KNOW.
ROHAN
Client
Call
11 AM
CLIENT TRUST
ACCURACY
COMMUNICATION


ROHAN WORLD
Guessing. Fast. Dangerous.
Hospital AI
Patient History
John D'Souza
Allergies
Penicillin
INCORRECT
Delivery Timeline
Client: NovaEdge
Committed Date
15 May 2025
WRONG
Actual Commitment
30 May 2025
Bank Chatbot
Home Loan Interest Rate
7.2%
Incorrect Information
HR Policy AI
Leave Policy
Earned Leave
30 Days / Year
FALSE
Company Policy
18 Days / Year
AI RESPONSE
CONFIDENT BUT WRONG
Sales Report - Q2
Total Revenue
₹28.4 Cr
INCORRECT
System Status
Unverified Data Sources
High Hallucination Risk
Low Accuracy
SPEED OVER ACCURACY
Not every confident answer is a correct answer.
ARJUN WORLD
Retrieval. Verified. Trusted.
Verified Policy Document
Leave Policy
Earned Leave
18 Days / Year
VERIFIED
Source: HR Portal
Policy Doc #HR-2024-19
Accurate Delivery Timeline
Client: NovaEdge
Committed Date
30 May 2025
VERIFIED
Source: Delivery System
Ref: DEL-2025-7781
Knowledge Retrieval
Top Relevant Sources
1. Meeting Notes - 12 May
2. Project Plan - v3.2
3. Client Contract
4. Email Thread
Sources Retrieved: 4
Lab Report - Verified
Water Quality Analysis
Status: Safe
Source: Lab Portal
Report ID: WQA-88421
AI RESPONSE
ACCURATE & TRUSTED
Confidence: High
Sources: 4
Accuracy: 98.7%
Contract Summary
Client: NovaEdge
Term: 12 Months
Value: ₹2.45 Cr
VERIFIED
Source: Contract Repository
Ref: CNTR-2025-556
ARJUN
TRUST OVER CONFIDENCE
DATA
KNOWLEDGE
WISDOM


The future of AI looks a lot like thoughtful humans.
Pause. Search. Retrieve. Respond.
SEARCH
PROJECT PLAN VERIFIED
MEETING NOTES MONDAY
DELIVERY TIMELINE CONFIRMED
KNOWLEDGE GRAPH
CITATION
Source: Project Doc v2.4
Updated: Monday
Verified: Yes
Confidence: High
RESPONSE
Accurate.
Relevant.
Trusted.
RETRIEVAL PATHWAY
FOCUS
COMMITMENT
DISCIPLINE
SUCCESS
Focus Creates Clarity
Retrieve Reality
ARJUN
DATA
KNOWLEDGE
WISDOM
IMPACT
THOUGHTFUL TODAY.
IMPACT TOMORROW.

RAG ARCHITECTURE

RETRIEVAL-AUGMENTED GENERATION

AI THAT THINKS WITH YOUR DATA

KNOWLEDGE INGESTION & INDEXING (OFFLINE)

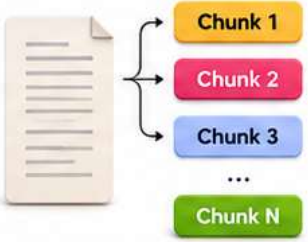
1 DOCUMENT SOURCE

HR Policy Documents (PDF, DOCX, TXT, etc.)

2 DOCUMENT LOADER

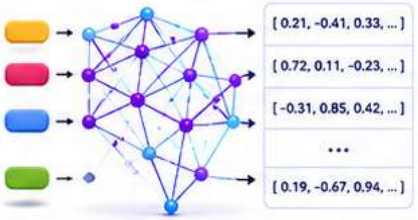
Load and extract text from documents

3 TEXT CHUNKING



Split text into smaller chunks

4 EMBEDDING MODEL



Convert chunks into vector embeddings

5 VECTOR STORE

Pinecone
ChromaDB
Weaviate
FAISS

Store embeddings in vector database

RAG PIPELINE (ONLINE QUERY FLOW)

1 USER QUERY

2 QUERY EMBEDDING

[-0.12, 0.47, 0.91, ...]

Convert user query into embedding

3 RETRIEVER

Search vector database for similar chunks

SIMILARITY SEARCH

Pinecone / ChromaDB
Weaviate / FAISS

4 TOP-K RETRIEVAL

Chunk A ★★★★
Chunk B ★★★★
Chunk C ★★★★
...
Chunk K ★★★★

Retrieve most relevant chunks

5 PROMPT AUGMENTATION

USER QUERY
+
RETRIEVED CHUNKS
+
SYSTEM INSTRUCTIONS

Combine query + retrieved context + system instructions

6 RAG AGENT (LLM)

Generate answer using retrieved context

7 FINAL RESPONSE

As per company policy, you can take 24 leaves in a year.

Verified Answer

WHY RAG WORKS

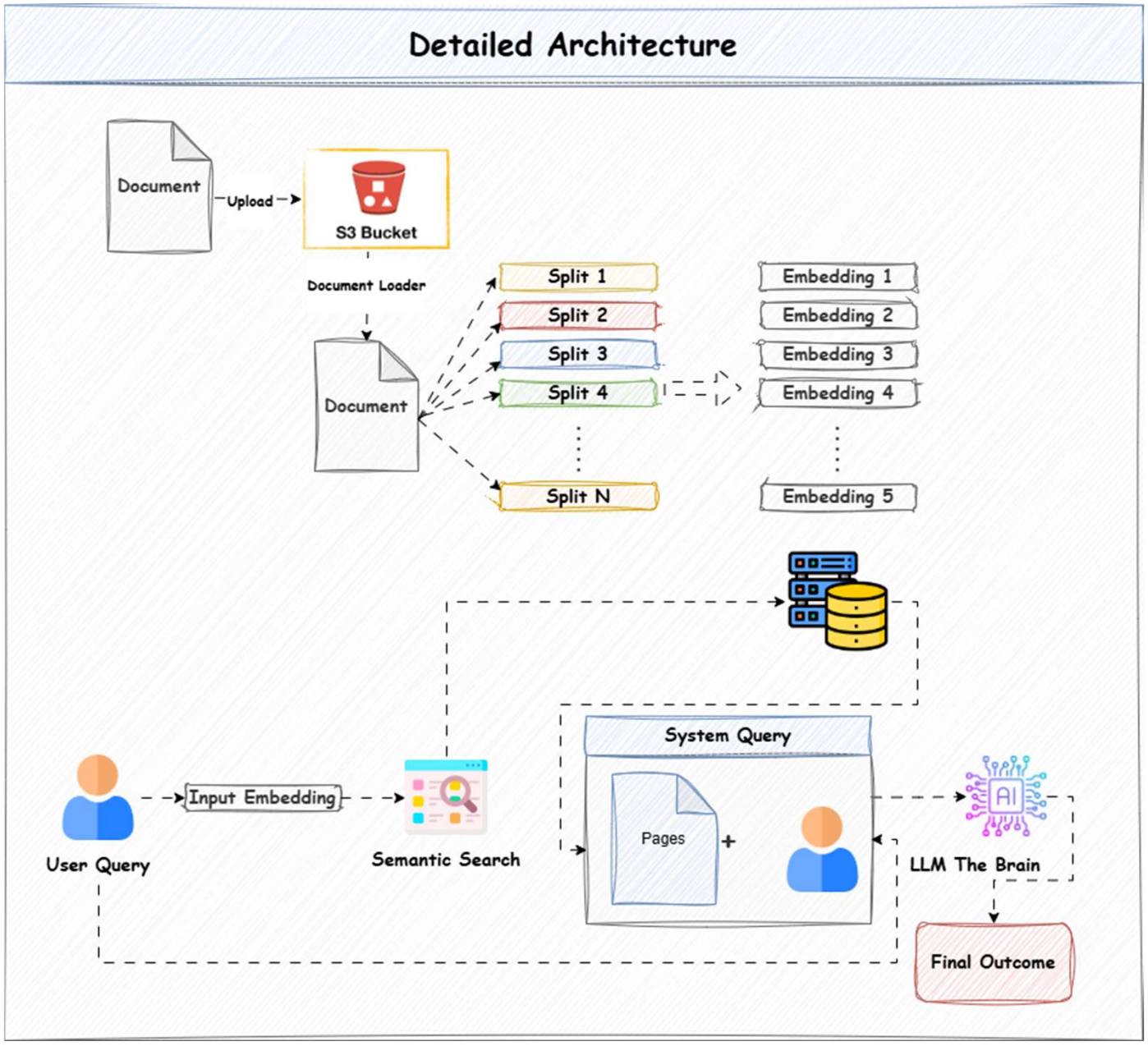
1. ACCURATE
Uses retrieved company data

2. RELEVANT
Finds most useful information

3. TRUSTED
Grounded on real documents

4. UP-TO-DATE
Reflects latest policies

5. EXPLAINABLE
Easy to trace information sources


Detailed Architecture
Document
Upload
S3 Bucket
Document Loader
Document
Split 1
Split 2
Split 3
Split 4
Split N
Embedding 1
Embedding 2
Embedding 3
Embedding 4
Embedding 5
System Query
Pages
User Query
Input Embedding
Semantic Search
AI
LLM The Brain
Final Outcome

WHY RAG EXISTS

The Problem With Standalone LLMs

Large language models are powerful — but they have real limits.

Knowledge Cutoff

An LLM only knows what it was trained on. It can't answer about today's news or last week's policy change.

Hallucinations

When it doesn't know, it makes things up — confidently. This is the #1 reason LLMs fail in production.

No Access to Your Data

It has never seen your company's invoices, contracts, internal docs, or customer history.

No Sources

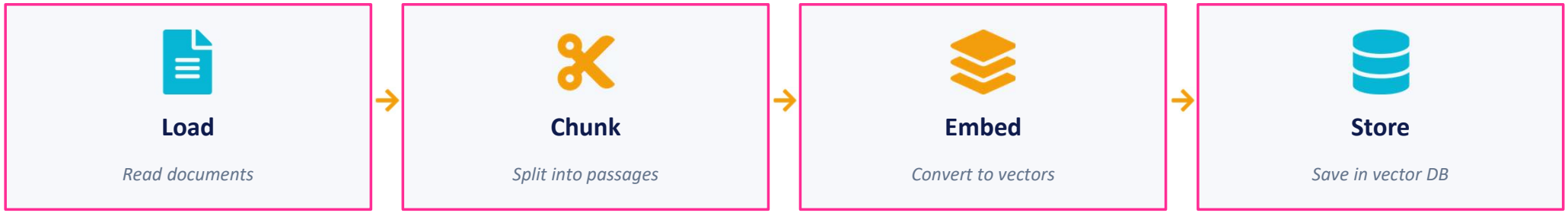
Plain LLMs can't show you where an answer came from — making them hard to trust for serious work.

THE PIPELINE

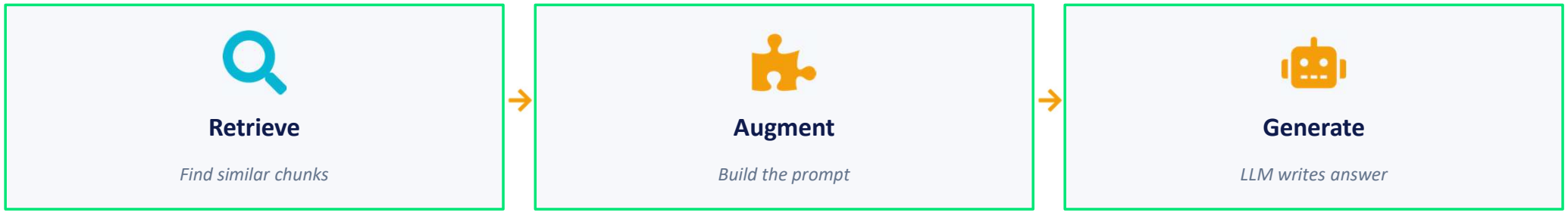
RAG Has Two Phases & Six Modular Stages

Get any stage wrong and the whole thing breaks. That's why we'll learn each one.

PHASE 1 — INDEXING (done once, in advance)



PHASE 2 — AT QUERY TIME (every time a user asks)



WHERE THINGS BREAK

Common Failure Points

Knowing where RAG fails is the first step to fixing it.

Loading	Bad text extraction from PDFs, missing structure, garbled encoding.
Chunking	Splits mid-sentence or mid-table. Retrieval can never recover what chunking destroyed.
Retrieval	Right chunk exists but isn't returned. Or the wrong chunks dominate top-K.
Augmentation	Context too long, badly ordered, or missing the key instructions to ground.
Generation	LLM ignores context, hallucinates anyway, or refuses to cite sources.

FORMATS

What You'll Be Loading From

Each format has its own quirks. Pick the right loader for the job.

PDF

Most common in enterprise. Text + tables + scans. Use PyPDF or PyMuPDF; OCR for scans.

HTML

Web pages and online docs. Strip navigation, ads, and boilerplate before chunking.

Markdown

Best case — already has headers and structure. Preserve them; they help retrieval.

DOCX / PPTX / XLSX

Office files. python-docx, python-pptx, openpyxl. Watch for embedded images.

APIs / JSON

Notion, Confluence, Slack. Use the official client; respect rate limits and pagination.

Plain Text

Logs, transcripts, code. Easy to load — but think hard about how to split.

BEST PRACTICES

Do's & Don'ts of Loading

These two columns separate a working RAG from a frustrating one.

DO

- Clean the text — strip ads, menus, boilerplate.
- Preserve structure — headings, lists, tables.
- Add metadata — source, page, date, author.
- Validate encoding — convert to UTF-8 early.
- Inspect output — eyeball the first few documents.

DON'T

- Dump raw bytes into the chunker — clean first.
- Ignore encoding errors — they cascade silently.
- Throw away metadata — you'll need it for filtering.
- Trust the PDF text layer blindly — scans need OCR.
- Skip inspection — bad input is invisible at query time.

THE PROBLEM

Why We Can't Just Embed Whole Documents

Two hard constraints force us to break documents into pieces.

Context Window Limits

LLMs have a max number of tokens they can read at once. A 500-page manual won't fit.

Retrieval Precision

If you embed a whole book as one vector, every query matches it equally. Useless.

⚠ What "chunking gone wrong" looks like

- Splitting in the middle of a sentence — the question's answer is now half in each chunk.
- Splitting a table from its header — numbers without column names are meaningless.
- One chunk = one whole chapter — too big, too noisy, low precision.
- 100-character chunks — too small, no context, every chunk looks alike.

STRATEGIES

Three Ways to Chunk — Fixed, Recursive, Semantic

Start with recursive. Move to semantic when quality matters.

Fixed-Size

Cut every N characters. Simple, fast, brain-dead.

GOOD

Easy to implement, predictable size.

BAD

Splits mid-sentence; ignores structure.

USE WHEN

Prototypes, toy demos.

Recursive

Split on paragraphs, then sentences, then words — until small enough.

GOOD

Respects natural boundaries; LangChain default.

BAD

Still blind to meaning across sentences.

USE WHEN

90% of real-world RAG starts here.

Semantic

Split where the meaning changes (using embeddings).

GOOD

Highest quality — chunks have one idea each.

BAD

Slower, more compute, harder to tune.

USE WHEN

When retrieval quality matters more than cost.

TUNING

Chunk Size vs Overlap

Two knobs. Get them right and retrieval gets dramatically better.

CHUNK SIZE

Small

100 – 300 tokens

✓ High precision, focused chunks

✗ Often missing context

Medium

500 – 800 tokens

✓ Balanced — good default

✗ Tune for your data

Large

1000 – 1500 tokens

✓ Lots of context per chunk

✗ More noise, lower precision

OVERLAP

Overlap = how many tokens chunks share at the boundary.

Why it matters: an answer that sits across a chunk boundary is lost without overlap. Start with 10–20% overlap (e.g., 600-token chunk → 100-token overlap). Too much overlap wastes storage; too little drops answers at boundaries.

FROM TEXT TO MATH

Embeddings — Meaning as Coordinates

An embedding turns text into a list of numbers that captures its meaning.

The Core Idea

Computers can't compare text directly.
But they can compare numbers — easily.

An embedding model reads a sentence and outputs a long list of numbers (usually 384–1536 of them). Sentences with similar meaning end up with similar numbers.

That's the entire trick. RAG retrieval is just: turn the query into numbers, find the chunks with the closest numbers.

Example: meaning as coordinates

"How do I reset my password?"

```
[0.12, -0.84, 0.31, …]
```

"My login isn't working."

```
[0.14, -0.81, 0.29, …]
```

"Best pizza in Naples?"

```
[-0.62, 0.05, 0.91, …]
```

First two are close in vector space → semantically similar.
The third lives in a different neighbourhood entirely.

MEASURING CLOSENESS

Cosine Similarity — How We Compare Embeddings

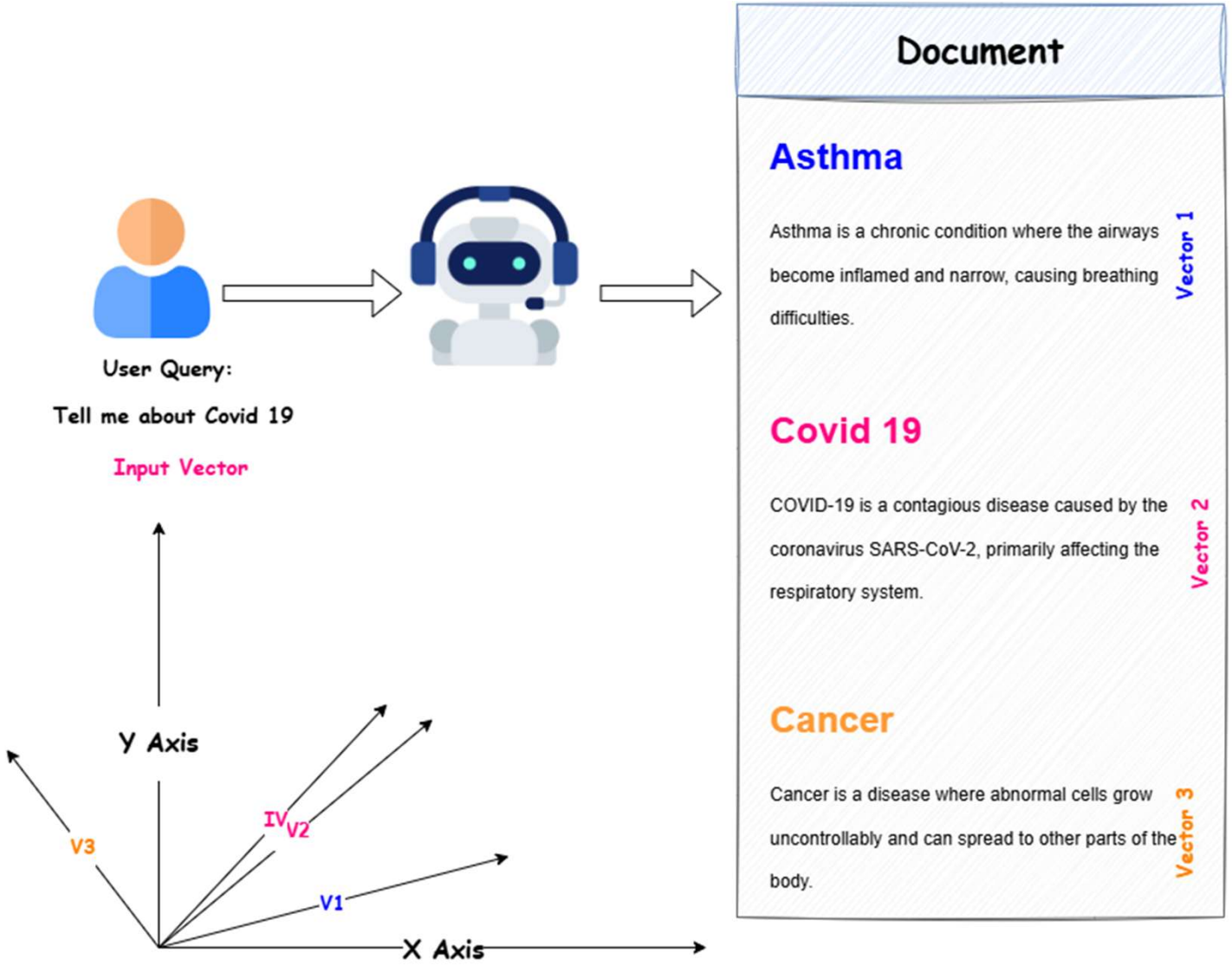
Once everything is a vector, we just measure the angle between them.

COSINE SIMILARITY — ranges from -1 to 1

1.0	0.85	0.5	0.2	-1.0
Identical	**Very Close**	**Related**	**Distant**	**Opposite**
Same meaning	Strong match — return this	Same topic, different angle	Probably unrelated	Very rare in practice

What is "top-K" retrieval?

After scoring every chunk against the query, we return the K chunks with the highest similarity scores. Typical K = 3 to 10. Smaller K = faster but might miss the answer. Larger K = safer but more noise in the prompt.


Document
Asthma
Asthma is a chronic condition where the airways become inflamed and narrow, causing breathing difficulties.
Vector 1
Covid 19
COVID-19 is a contagious disease caused by the coronavirus SARS-CoV-2, primarily affecting the respiratory system.
Vector 2
Cancer
Cancer is a disease where abnormal cells grow uncontrollably and can spread to other parts of the body.
Vector 3
User Query:
Tell me about Covid 19
Input Vector
Y Axis
X Axis
V1
V2
V3
IV

Cosine Similarity — Mathematical Explanation

WHERE EMBEDDINGS LIVE

Vector Stores — A Quick Tour

Specialised databases optimised for fast similarity search over millions of vectors.

FAISS

Local

Facebook's library. Local, fast, no server. Best for learning and prototypes.

ChromaDB

Local

Embedded vector DB in Python. One-line setup. Great default for course work.

Pinecone

Cloud

Managed cloud service. Scales to billions of vectors. Pay-as-you-go.

Weaviate / Qdrant

Both

Open-source, production-ready. Run self-hosted or managed.

pgvector

DB add-on

PostgreSQL extension. If you already use Postgres, this is the easiest path.

Milvus

Cloud

Built for very large scale. More setup, more power. Used by big tech.

RETRIEVAL TECHNIQUES

Dense, Sparse, and Hybrid Search

Use both — they make different mistakes, so they cover each other.

Dense (Semantic)

Embedding-based. Finds chunks that mean the same thing — even with different words.

Example: Query "cheap car" finds chunks about "affordable vehicle"

Sparse (BM25 / Keyword)

Classic keyword search. Great when the exact term matters — product codes, names, technical jargon.

Example: Query "ISO 27001" finds documents that literally mention "ISO 27001"

Hybrid (Both)

Run both retrievers, fuse the results. Catches what each one misses.

Example: Best default for production — start here when accuracy matters

RE-RANKING

Retrieve Many, Then Rerank

The single biggest precision boost you can add to RAG.

STEP 1

Retrieve 20 chunks

Fast but imprecise — bi-encoder similarity

STEP 2

Rerank with cross-encoder

Slower but much more accurate — query + chunk scored together

STEP 3

Keep top 3

Highest-quality context for the LLM

The Retrieval Principle

Optimise recall first, then precision.

Get the right chunk into your top 20 (recall). Then use reranking to push it into your top 3 (precision). Don't rely on top-K similarity alone — bi-encoder scores are fast approximations, not ground truth.

Tools: Cohere Rerank (managed) · BGE Reranker (open-source) · LLM-as-judge (use the LLM itself to score)

AUGMENTATION

Retrieval vs Augmentation — They're Different Jobs

Retrieval finds the chunks. Augmentation decides how to use them.

RETRIEVAL	AUGMENTATION
Question: Which chunks are relevant?	**Question: How do I give them to the LLM?**
Job: Search the vector store and return the top-K most similar chunks.	Job: Build a prompt with instructions, retrieved context, and the user question.

The Augmentation Trade-Offs

How much context to include? Too little → LLM misses facts. Too much → LLM gets confused, latency rises, cost climbs.
How to order chunks? Most-relevant first or last? The "lost in the middle" problem is real.
How to structure prompts? Numbered, separated, with metadata? Small changes have big effects.

ANATOMY

What's Inside a RAG Prompt

Four parts. Each one matters. Skip any of them and quality drops.

1

System Instructions

Defines the assistant's role and rules. "You are a helpful assistant. Answer only from the context. If unsure, say 'I don't know'."

2

Retrieved Context

The top-K chunks, usually labelled with their sources. "Source 1 (page 42): ... / Source 2 (page 87): ..."

3

The User Question

Placed clearly, often after the context. "Question: What is our refund policy?"

4

Output Instructions

Format and citation rules. "Cite the source number for every claim. Answer in two paragraphs maximum."

PATTERN 2 & 3

Map-Reduce and Refine — When Context Doesn't Fit

Two patterns for processing more chunks than the LLM can read at once.

MAP-REDUCE

1 Map

For each chunk, ask the LLM: "What does this chunk say about the question?"

2 Reduce

Combine all the per-chunk answers into one final answer with a second LLM call.

✓ Parallel — fast for many chunks

✗ Loses cross-chunk context

REFINE

1 Initial answer

Ask the LLM for an answer using just the first chunk.

2 Improve, chunk by chunk

For each next chunk: "Here's the answer so far + a new chunk. Improve the answer."

✓ Preserves cross-chunk context

✗ Sequential — slower, more LLM calls

GROUNDING

Making the LLM Stay on the Page

These prompt techniques reduce hallucinations dramatically — and they cost nothing.

Explicit grounding instruction

"Use ONLY the information in the context below. Do not use prior knowledge."

Force "I don't know"

"If the answer is not in the context, reply exactly: 'I don't know based on the provided documents.'"

Citation requirement

"For every fact, cite the source number in square brackets, like [Source 2]."

Chain-of-thought hint

"First identify which sources are relevant, then synthesize the answer step by step."

SHOW THE DIFFERENCE

With vs Without Structured Prompts

Same chunks, same model. The only thing that changed is the prompt.

WITHOUT ("answer this question")

"Refunds are typically issued within 7–14 business days. We offer a satisfaction guarantee on most products. Contact our support team for assistance with your refund."

Problems:

- Made up numbers (7–14 days isn't in our docs)
- No citation — can't verify any of it
- Generic boilerplate, not policy text

WITH (grounded + cite + "I don't know")

"Refunds are available within 30 days of purchase [Source 1, page 42]. The product must be unused and in original packaging [Source 1, page 43]. Digital goods are non-refundable [Source 2, page 7]."

Wins:

- Every fact has a source — verifiable
- No hallucinated numbers
- Same model — different result

WHY IT'S HARD

RAG Evaluation Is Two Problems, Not One

You're not just grading an LLM. You're grading retrieval and generation separately.

Did retrieval find the right chunks?

If the right info never reached the LLM, no amount of prompt tuning will fix the answer.

Did the LLM use them correctly?

Even with perfect retrieval, the LLM can ignore context, misread it, or hallucinate anyway.

Why you can't just count "right answers"

Real RAG answers are open-ended, often partially correct, sometimes phrased differently from your reference. "Did the LLM cite Source 2?" is binary. "Is this paragraph faithful to the context?" is a judgment call. RAG evaluation mixes hard metrics (retrieval) with soft judgments (generation).

RETRIEVAL METRICS

How to Score Your Retriever

These four metrics tell you whether the right chunks are coming back.

Precision@K

Of the top K chunks returned, what fraction are actually relevant?

Example: Top-5 has 3 relevant chunks → Precision@5 = 60%

Recall@K

Of all relevant chunks in your corpus, what fraction made it into the top K?

Example: 3 relevant exist, 2 returned → Recall@5 = 67%

MRR (Mean Reciprocal Rank)

How high in the ranked list is the first relevant chunk?

Example: First hit at position 3 → 1/3 = 0.33

NDCG (Normalised DCG)

Rewards relevant chunks more if they appear higher in the ranking.

Example: Used when ordering matters — usually it does

GENERATION METRICS

How to Score Your Answers

Three signals to track separately — they fail independently.

Faithfulness

Does the answer stick to what's in the retrieved context — or invent things?

→ Hallucinations live here

Answer Relevance

Does the answer actually address the user's question — or drift off-topic?

→ Off-topic answers live here

Answer Correctness

Is the answer factually right, judged against ground-truth labels?

→ Real-world accuracy lives here

FRAMEWORK

RAGAS — The Industry-Standard RAG Evaluator

Open-source. Uses an LLM as judge. Four core metrics that map cleanly to what we've covered.

Context Precision

Are the retrieved chunks ranked well — relevant ones near the top?

Context Recall

Did retrieval bring back enough of the relevant information for the answer?

Faithfulness

Is the generated answer supported by the retrieved context?

Answer Relevance

Does the answer actually address the question that was asked?

In practice: pip install ragas → feed it questions, answers, retrieved contexts, ground truths. It scores each metric 0–1 using an LLM judge.

BUILDING THE DATASET

The Hardest Part of Evaluation

Without good test data, every metric is meaningless. Spend time here — it pays back.

Ground-truth Q&A pairs

Real questions with verified correct answers and the chunks they came from. Aim for 30–100 to start.

Negative testing

Questions whose answers AREN'T in your corpus. Make sure your RAG says "I don't know" instead of inventing.

Metrics vs rubrics

Metrics = automatic numbers (faithfulness 0.84). Rubrics = qualitative checklists (does it cite? does it hedge?). Use both.

Real user questions

Synthetic test sets miss the weird ways real users ask. Mine your actual logs once you have any.

FAILURE MODES

Where RAG Quality Actually Breaks

Most production RAG failures come from one of these. Knowing them shortcuts debugging.

Retrieval miss	The right chunk exists in the index but didn't make top-K. Fix: better chunking, hybrid search, reranking.
"Lost in the middle"	Right chunk was retrieved but buried in the middle of the prompt — LLM ignored it. Fix: place key chunks at start or end.
Hallucination despite context	LLM had the answer but made up something else. Fix: stronger grounding prompts, lower temperature.
Context window overflow	Top-K chunks together exceed the LLM's context. Fix: smaller chunks, fewer K, or switch to map-reduce.
Stale index	Docs were updated but the index wasn't re-built. Fix: scheduled re-indexing jobs, monitor freshness.

MANUAL VS AUTOMATED

You Need Both — Here's Why

Automated eval is fast and cheap. Manual eval catches what automation misses.

AUTOMATED (RAGAS, scripts)

✓ Fast — score 100 questions in minutes
✓ Cheap — runs on every code change
✓ Reproducible — same input, same score

✗ LLM-judge can hallucinate scores
✗ Misses tone, style, edge cases
✗ Calibrates to its training, not your users

MANUAL (human reviewers)

✓ Catches subtle errors automation misses
✓ Calibrates to your users' expectations
✓ Surfaces problems you can't define yet

✗ Slow — minutes per question
✗ Expensive at scale
✗ Inter-reviewer disagreement is real

THE EVOLUTION

Naive → Advanced → Modular RAG

Three generations of RAG architecture. Each adds more control points.

Naive RAG

What we built today. Embed, retrieve, stuff, generate.

Good for most starting use cases.

Advanced RAG

Adds pre-retrieval and post-retrieval steps — query rewriting, reranking, contextual compression.

Standard for production systems.

Modular RAG

Treats every stage as a swappable module. Routing, hybrid retrieval, multiple LLMs, feedback loops.

Where serious RAG engineering lives.

BEYOND VECTOR SEARCH

Where RAG Goes Next

Each of these solves a class of problems naive RAG can't.

Agentic RAG

RAG that decides — when to search, what tool to use, when to re-try. Becomes a small agent.

Multi-hop RAG

Breaks complex questions into sub-queries, retrieves for each, combines the answers.

Graph RAG

Retrieval over a knowledge graph. Best when relationships between entities matter.

Adaptive Retrieval

Dynamically picks K, rewrites the query, or routes to different indexes based on the question.

Long-Context vs RAG

Modern LLMs have huge context windows. Sometimes you can just stuff the whole doc — but it costs more.

Caching Strategies

Cache embeddings, retrievals, and full responses. Cuts cost and latency dramatically.

WHEN TO LEVEL UP

A Practical Heuristic

Don't add complexity until you've measured a real problem with the basics.

The Order of Operations

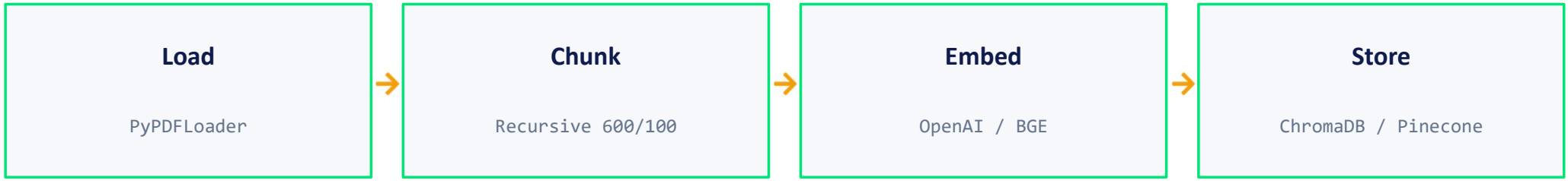
1. Build naive RAG. Get it working end-to-end before tuning anything.
2. Build a small eval set (30 questions, ground-truth answers). Measure baseline.
3. Fix retrieval first — it's where most quality is won. Better chunking, hybrid search, reranking.
4. Then improve generation — grounding prompts, citations, refine pattern for long contexts.
5. Only after that, consider agentic / graph / multi-hop. Most projects never need them.

THE FULL PICTURE

Everything We Built Today

The complete RAG pipeline, one more time — but now you've seen every piece.

INDEXING



QUERY TIME

Eight stages. Each one a place to make your RAG noticeably better.

DEBUGGING WORKFLOW

When Your RAG Gives a Bad Answer

A simple flow to find what broke — every time.

1

Did the right chunks get retrieved?

→ Print top-K results. If the answer chunk isn't there → fix chunking, embedding, or retrieval.

2

Were the chunks readable in context?

→ Look at the final prompt. Garbled text, weird ordering, lost-in-the-middle? Fix augmentation.

3

Did the LLM follow the instructions?

→ If chunks were good but the answer ignored them → strengthen grounding, lower temperature.

4

Is this a one-off or a pattern?

→ Run RAGAS on your eval set. If scores dropped, find the metric that moved — that's your root cause.

THAT'S A WRAP

You Now Speak RAG.

You can load any document, chunk it sensibly, build a vector index, retrieve grounded context, prompt the LLM properly, and measure whether the whole thing actually works.

That's a real, deployable skill. The rest is reps.

Questions?